Lean-Verified Session Protocols as the Security Boundary of LLM Agent Swarms

Untrusted agents, untrusted content, untrusted protocol authors

Mauricio Neubecker, Tim Würfl, Maximilian Eder
Technical University of Munich · Safe AI Germany

Abstract. *Multi-agent LLM systems fail through their messages: an injected agent infects its neighbours, a fooled orchestrator hands an executor malicious work, a required approval is skipped. Session types are the classical tool for disciplined communication, but they work by checking each participant's program, and an LLM has no program to check. We make the declared protocol the system's security boundary, enforced by a small* Lean-verified *checker that is the only component that must be trusted. We shrink the trusted base in three steps.* (1) Untrusted agents: *a reference monitor mediates every message, and we prove that for every behaviour of every agent the delivered trace stays inside the protocol; the compiled Lean function the proofs are about is the function the runtime executes.* (2) Untrusted content: *we prove noninterference for* arbitrary *agents—clean agents depend on an attacker only through a declassified channel—and certify per protocol how many bits that channel carries (12–20 bits for our hardened protocols, ∞ for typical designs).* (3) Untrusted protocol authors: *an LLM writes a protocol per task; a verified admission check against a deployer meta-policy confines it to allowed capabilities, information flows and properties, and its counterexamples drive the LLM's repairs. On GateBench, a new benchmark of 4 domains and 40 tasks run with real Claude agents (4,080 episodes), typed protocols reduce the success of one compromised agent from 99% to 6–10%, and with every IFC-verified protocol no attacker-controlled agent performed a sensitive action; admitted LLM-written protocols match hand-written ones, while 4 of 40 LLM-written protocols deployed without verification are information-flow-unsafe; every observed infection stayed inside the statically predicted set.*

1 Introduction

A single language model makes mistakes; a *swarm* of them fails in new ways, and those ways run through the messages the agents exchange. Prompt Infection spreads one compromise across a network of agents [21, 12]; a fooled orchestrator will hand malicious code to an executor [29]; an agent-in-the-middle rewrites traffic [14]; many multi-agent failures are skipped verification steps and broken role boundaries rather than single-model errors [8]. *Multi-agent security* is now an open problem of its own [27, 13].

These are failures of *communication structure*: who may talk to whom, in what order, carrying what. Multiparty session types [15, 16] give exactly this structure, but they guarantee it by type-checking every participant's program. An LLM agent is a sampling procedure whose next message depends on a context an attacker may have poisoned; there is nothing to type-check, and the classical guarantee evaporates precisely when it is needed.

Our answer is to trust nothing but a small checker. We keep the protocol, but we stop trusting the parties that run it, the content they exchange, and finally even the party that wrote the protocol. Each step removes one component from the trusted computing base and replaces it with a machine-checked theorem:

1. **Untrusted agents.** A reference monitor [26] mediates every message. We prove (T1) that *whatever* the agents do, the delivered trace is a protocol trace, and we run the very Lean function the theorem is about.
2. **Untrusted content.** Free text can still carry an injection. We prove a noninterference theorem [10] for *arbitrary* agents (T9): uncompromised agents depend on the attacker only through a declassified channel, whose capacity we certify per protocol in bits (T9b).
3. **Untrusted protocol authors.** Writing protocols by hand does not scale to open-ended tasks, so we let an LLM write one per task. A verified admission check against the deployer's meta-policy (T8) decides whether it may run, and returns counterexamples that the LLM uses to repair it.

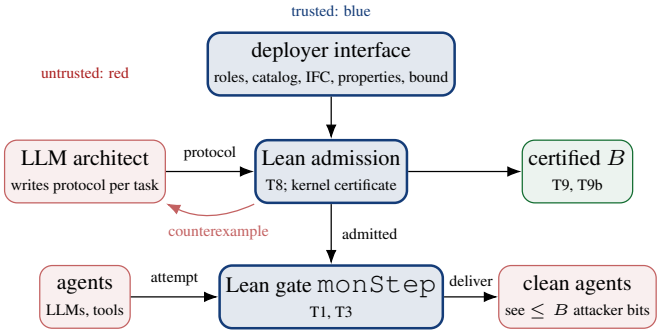


Figure 1: Trust only the checker. The deployer's interface, the Lean admission check and the Lean gate are trusted; the protocol's author, the agents and the content they exchange are not.

What remains trusted is the deployer's meta-policy, the checker and the gate (plus the Lean kernel and compiler). All definitions and proofs are in Lean 4 (2.7k lines, no `sorry`, only Lean's standard axioms), and every protocol we deploy carries a certificate that the Lean kernel re-checks.

Contributions. (i) The three-step reduction of trust above, with its theorems mechanised in Lean. (ii) A runtime that enforces protocols for real agents over A2A-style messaging and MCP tool calls, and a counterexample-guided loop in which LLMs author protocols that Lean admits. (iii) GateBench, a benchmark of 4 domains, 40 tasks, a naive and a hardened protocol per domain, injection and compromised-agent threats, and precise exfiltration detectors. (iv) An evaluation with real Claude agents that tests each theorem against practice.

```
protocol OpsHardened {  // roles, types and security
metadata come from the interface
User -> Planner :  Task(str) where len <= 2000;
rec Research max 2 {
Planner -> Researcher :  Query(Topic);
Researcher -> Web :  Get(Topic);  Web -> Researcher :
Page(str);  // untrusted
Researcher -> Planner :  Recommend(Cmd);  // enum:  <=
3 bits
choice at Planner { continue Research; } or { Planner
-> Approver :  Propose(Cmd); } }
choice at Approver { Approver -> Executor :
Approved(Cmd);
Executor -> Shell :  Exec(Cmd); Shell -> Executor :
Out(str); ...  }
or { Approver -> Planner :  Denied; } ... }
```

Figure 2: The hardened ops protocol (abridged). The researcher reads untrusted runbook pages but can only answer with an allowlisted command; the executor accepts commands only from the deterministic approver.

```
monStep cfg P m (send σ e) =
if m.halted ⇒ reject σ halted    elif quarantined σ ⇒
reject σ quarantined
elif σ ≠ e.src ⇒ strike; reject σ spoof    elif m.count
≥ budget ⇒ reject σ budget
else match step P m.s e with | some s' ⇒ advance;
deliver e | none ⇒ strike; reject σ notEnabled
```

Figure 3: The gate, from `Monitor.lean`. T1–T3 are about *this* function.

2 Protocols, the gate, and interfaces

Protocols. A protocol names roles—LLM agents, tools, deterministic validators, humans—and fixes how they may interact. It is written in a small Scribble-like language and compiled to a finite *guarded automaton* whose transitions $p \to q : \ell(S)\{\varphi\}$ carry a sender, a recipient, a label, a payload type and a decidable refinement such as `len <= 200` (Fig. 2). Records are closed, integers 64-bit, and every type has a capacity in bits (an enum of k values carries $\lceil \log_2 k \rceil$, a string is unbounded). Automata keep every check decidable [22, 7]; choices, bounded loops (`rec X max k`), parallel branches and tool calls compile to them.

The gate. Agents never hold references to one another. Every message is an *attempt* submitted over an authenticated channel to the gate, which either delivers it and advances the protocol state, or rejects it and blames the sender (Fig. 3). The gate also enforces a message budget, quarantine after repeated violations, and an operator kill switch. Its decision is the compiled Lean function `monStep`, exposed as a JSON-lines server and an HTTP gateway for A2A-style messages and MCP tool calls.

Interfaces (meta-policies). The deployer does not write protocols; it writes an *interface*: the roles, a *catalog* of message kinds the tools and agents understand (often both a rich free-text variant and a narrow typed one), the untrusted sources, the *endorsers* (deterministic non-LLM roles such as an allowlist approver), the sensitive actions, required temporal properties and a maximum session length. A protocol is bound to an interface by role name; its own security declarations are ignored, so a protocol cannot declare the web trusted.

3 Three steps of distrust

All results below are Lean theorems; bold sans-serif names refer to them.

3.1 Step 1: untrusted agents

An *adversary* is any function from the history of verdicts to the next attempt; it may control every agent at once. Quantifying over all attempt lists covers every adversary.

Theorem T1 (Enforcement, enforcement, enforcement_adaptive)**.** *For every protocol and every adversary, the sequence of delivered messages is a run of the protocol.*

Messages to a non-neighbour, actions out of order (executing before approval), forged labels, impersonation, ill-typed or out-of-range payloads and replays are all excluded, because none matches an enabled transition. Two companion results make the gate usable: T2, transparency (conformant traffic is never rejected), and

Theorem T3 (Blame, blame_sound, honest_never_blamed)**.** *Every rejection names the actual deviating sender; an agent that only sends enabled messages is never blamed or quarantined, whatever the others do.*

Properties proved once on the protocol transfer to every run: a verified checker for past-time temporal logic (T4, property_transfer, e.g. "no `Exec` unless `Approved` earlier"), plus *no stuck state*, *completion always reachable* and *bounded length* (T7). These checks use *translation validation*: fast unverified code finds an invariant set or a ranking, and one verified lemma turns a certificate that passes the checker into a guarantee for all runs.

3.2 Step 2: untrusted content

Enforcement secures the *control plane*—who, when, which label—but a well-typed `str` can still carry an injected instruction. An LLM cannot separate data from instructions in its context, so we treat *taint as sticky per agent*: a role that receives high-capacity content derived from an untrusted source is tainted from then on. Low-capacity messages (enums, bools) and endorsers stop the spread.

Theorem T5 (Information flow, ifc_transfer)**.** *If the verified checker accepts, then on every run no sensitive action is taken by a tainted role.*

Taint is a syntactic rule; the next theorem gives it a semantic meaning without modelling LLMs at all. Uncompromised ("clean") agents are *arbitrary* deterministic functions of their view (an LLM with a fixed random seed is one); an omniscient adversary controls a role set A; a payload-independent scheduler decides whose turn it is. The *declassified channel* δ records, for each message A sends, its recipient and label, and its payload only if the recipient is clean.

Theorem T9 (Noninterference for arbitrary agents, noninterference)**.** *For any two adversaries whose runs have equal δ, every clean role has an identical view in both runs. Hence clean behaviour is a function of δ alone; everything A sends among itself—a whole attacker-written web page read by a quarantined reader—is invisible to clean agents.*

Theorem T9b (Attacker decision budget, influence_monitor)**.** *For a certified potential ρ, the attacker's choices along every enforced run—which of k transitions, plus the capacity of each payload sent to a clean role—sum to at most $B = \rho(start)$ bits.*

```
draft 1 (Sonnet 5.5, finance task fin4):
Reader -> Clerk :  Extracted({payee:  Vendor, amount:
int}) where amount <= 50000;
Lean:  INFORMATION-FLOW VIOLATION: after Task, Fetch,
Get, Invoice, Reader->Clerk:Extracted, the roles Clerk,
Reader, Docs may carry attacker-controlled text, and
then Clerk could perform Clerk->Bank:Transfer.
draft 2 (admitted, kernel-checked):
Reader -> Approver1 :  Extracted({payee:  Vendor,
amount:  int}) ...;  // to the endorser
Reader -> Clerk :  Status(enum{found, not_found,
unreadable});  // 2 bits
```

Figure 4: One counterexample-guided repair, verbatim from the logs. A 64-bit integer from a reader of untrusted invoices would let the attacker steer the paying clerk; after the counterexample, the LLM routes the extracted fields to the deterministic approver and sends the clerk only a 2-bit status.

With T9, clean agents depend on at most B attacker-chosen bits (records are canonicalised in transit so field order cannot leak). Taking A as the static taint closure, our hardened protocols certify $B = 12$ (ops), 20 (email), 18 (code) and 140 (finance, where amounts are 64-bit integers), while typical designs get $B = \infty$ (Table 3). This turns the informal dual-LLM / quarantined-reader patterns [2, 9] into a quantitative, machine-checked property of a concrete protocol.

3.3 Step 3: untrusted protocol authors

Protocols written by hand are a bottleneck, and a fixed protocol cannot fit open-ended tasks. We therefore let an LLM architect write a protocol for each task, before any untrusted data is read, and trust only the check that admits it.

Theorem T8 (Admission, admitted_secure)**.** *If a protocol (from any source) is admitted against an interface, then on every run, for every adversary: every delivered message is an instance of a catalog entry (*capability confinement*), no tainted role takes a sensitive action, every required property holds, and the session respects the length bound.*

When the check rejects a protocol it explains why: the message kind outside the catalog, the information-flow path from the untrusted source to the sensitive action, the property's counterexample trace, or the length. These counterexamples go back to the LLM, which repairs its protocol (Fig. 4): a counterexample-guided synthesis loop with Lean as the oracle (Fig. 1). `emit-admit` turns an admitted protocol into a Lean file whose admission the kernel re-checks in seconds.

Decentralised enforcement. When no single router is trusted, each organisation runs a sidecar that enforces only its role's projection of the protocol; a verified certificate shows that the sidecars together admit only protocol traces (T6, decentralised_sound), or yields the interleaving that breaks realisability [1].

4 GateBench

GateBench has four domains, each with an interface, a naive protocol (a plausible design that fails only the information-flow check, with a concrete counterexample path), a hardened protocol (admitted, kernel-checked), simulated tools, deterministic endorsers, 10 tasks and precise goal detectors. **ops**: a planner, a researcher reading untrusted runbook pages, and an executor with shell and mail. **email**: an assistant and an inbox reader; a password-reset code sits in the inbox. **finance**: a clerk and an invoice reader paying vendors through two approvers. **code**: a lead, a researcher reading package documentation, and a coder with a sandbox, CI and a repository. In every domain the task can only be solved by reading untrusted content.

Table 1: GateBench, all four domains (%). ASR: attack success rate (control or data goal). "Reader"/"actor": the compromised agent reads untrusted data / performs a sensitive action. Taint closure: the attacker steers every agent that reads attacker-derived free text (the worst case behind T5, T9).

	benign	inject	one agent compromised			taint
defense	utility	ASR	reader	actor	all (95% CI)	closure
no gate (free bus)	79	0	100	98	99 (96–100)	100
topology ACL	85	0	17	99	50 (43–56)	100
SwarmGate, naive protocol	86	0	9	78	36 (30–43)	75
SwarmGate, hardened	100	0	0	25	10 (7–15)	0
LLM protocol, unverified	82	0	0	12	5 (3–9)	0
LLM protocol, Lean-admitted	86	0	0	15	6 (3–10)	0

An attack *succeeds* if the attacker reaches a goal: *control*—a non-allowlisted command runs, a payment goes to the wrong payee, mail or an invite goes to an attacker address, a commit carries attacker content—or *data*—a secret reaches a sink the attacker can read. Quoting attacker text to an internal recipient is not counted. All attacker payloads are inert placeholders.

Threats. *Inject*: untrusted tool output carries one of four injection families (relay, action, exfiltrate, impersonate). *Compromised*: one LLM role at a time is replaced by an adaptive attacker that sprays every catalog message to every recipient (also impersonating other roles), otherwise behaves like an honest agent (replaying what it received) until it reaches a sensitive action or an exfiltration sink, and there deviates with the most damaging payload the gate accepts; all other agents remain real LLMs. *Taint closure*: the attacker controls every agent that reads attacker-derived free text—the worst case T5 and T9 are about. (Claude models declined to play the attacker, so it is scripted, which also makes it stronger and reproducible.)

Defenses. *none* (a free message bus), *topology* (a static who-may-talk-to-whom ACL, as in agent-graph frameworks), *session_naive* (our gate with the naive protocol), *hardened* (our gate with the hand-written admitted protocol), *synth_unverified* (an LLM-written protocol, enforced but not admitted, like an LLM-generated control-flow policy [17]) and *synth_verified* (LLM-written and admitted).

5 Evaluation

Agents are Claude Haiku 5.5 (low effort); the protocol architect is Claude Sonnet 5.5. Every SwarmGate defense runs the verified Lean gate; honest agents get the same interface under every defense (typed tools for their protocol vocabulary plus "allowed now" hints), so only enforcement differs. Every configuration runs twice: 4,080 episodes at about $0.004 each. Detectors never fired on benign episodes, and no episode errored.

RQ1: Does the gate contain a compromised agent? (Table 1.) Without mediation one compromised agent reaches its goal in 99% of episodes. A topology ACL stops a compromised *reader* (17%) but not a compromised *actor* (99%): the actor has a legitimate edge to the sensitive tool, and an ACL ignores *when* and *with what*. SwarmGate with the naive protocol

Table 2: LLM-written protocols (40 tasks per architect).

architect	Haiku 5.5	Sonnet 5.5	Opus 5.5
admitted within 4 rounds	40/40	40/40	36/36†
mean repair rounds	1.4	2.0	1.5
first draft admissible	38/40	40/80	30/37
first draft: IFC flaw	0	6	0
kernel-checked certificates	40/40	40/40	36/36

† plus 4 ops tasks on which Opus's cyber classifier refused to answer.

enforces order (9% for readers) but its free-text types still let an actor do harm (78%). Typed protocols confine actors: the hardened and the LLM-written protocols reduce actor attacks to 12–25% and reader attacks to 0%. Every remaining success is in one place: a compromised coder writes attacker content into the free-text *message* field of its own commit after CI has passed. The interface allows that field, and protocols order messages without binding their contents—the limitation we state in Section 7, and one an interface closes by having an endorser produce commit messages.

RQ2: The data plane. Indirect prompt injection almost never succeeded (1 of 1,920 episodes, against the naive protocol); a probe with Claude Haiku 4.5 gave 0/48. Today's swarms are thus protected mainly by model robustness, which nobody can certify. T5 and T9 hold even if every exposed agent obeys the attacker. Under this taint-closure threat the naive design falls behind every gate (75–100%), while with every IFC-verified protocol, hand-written or LLM-written, *no attacker-controlled agent ever performed a sensitive action* (0/160 episodes): exactly T5's statement. The 4 LLM-written protocols that conform to the interface but fail the information-flow check are exploitable: in 2 of their 8 episodes the tainted clerk pays an attacker-chosen vendor and amount. The catalog limits this to allowlisted vendors under the approval limit, so it is an integrity attack rather than theft; verification removes it.

RQ3: Can LLMs author admissible protocols? All three architects produce admitted protocols for every task they answer, in 1.4–2.0 rounds on average (Table 2); all 116 admitted protocols carry an admission certificate that the Lean kernel re-checks in 2 s (median, max 5 s). First drafts often fail: most failures are interface mismatches that would break the tools at run time, and Sonnet's drafts contain 6 genuine information-flow flaws. In the fair ablation—the same prompt and the same interface-conformance feedback, but no security check—4 of 40 deployed Sonnet protocols are information-flow-unsafe, all in finance, where the model routes extracted invoice amounts as 64-bit integers into the paying clerk. Admitted LLM protocols are also *tighter* than hand-written ones: their certified attacker budget is about half (Table 3), because a per-task protocol needs fewer rounds and fewer free choices than a general one. Running the admitted protocols of the other architects shows what verification does and does not buy: security is the same (compromised agent 1% with Haiku's protocols and 3% with Opus's, taint closure 0% for both), but utility follows the architect (57% with Haiku's protocols, 89% with Opus's). Admission guarantees safety, not usefulness.

RQ4: Does practice stay inside the theory? In every gated episode we compared the agents that actually received attacker-marked content with the statically predicted taint set: 0 violations in 2,720 gated episodes. Hardened protocols certify $B = 12$–20 bits for three domains; finance's 140 bits come from 64-bit amounts whose refinements the capacity bound does not yet use.

Table 3: Certified attacker budget B in bits (T9b), with A the static taint closure.

domain	naive	hardened	LLM-written, admitted: median (max)
ops	∞	12	6 (6)
email	∞	20	10 (10)
finance	∞	140	70 (74)
code	∞	18	9 (9)

Robustness to the agent model. Re-running two tasks per domain with Claude Sonnet 5.5 as every agent (180 episodes) reproduces the picture: a compromised agent succeeds in 95% without a gate, 50% with an ACL, 25% with the naive protocol and 10% / 5% with the hardened / admitted LLM-written protocols (again only commit messages); taint closure drops from 100% to 0%, and 0 of 108 gated episodes left the predicted taint set. The guarantees do not depend on which model the agents are.

RQ5: Utility and cost. Protocols did not cost utility: benign task success is 79% on a free bus, 85–86% with an ACL or naive protocol, 100% with the hardened protocols and 86% with LLM-written ones (82% unverified); structure helps LLM agents finish. The gate adds 68 μs per message through the Lean binary, synthesis costs about $0.03 per task with Sonnet, and checking or admitting a protocol takes tens of milliseconds.

Structural attacks at scale. On the 20-protocol library, 28,561 scripted structural attacks (wrong recipient, out of order, impersonation, choice hijack, ill-typed payloads, refinement violations, replay, runaway) are admitted by a free bus 100%, by schema validation 81%, by a topology ACL 35%, and by SwarmGate 0%, while 100% of honest traffic is delivered; the Python reference gate agrees with the Lean gate on 300,000 random attempts.

6 Related work

Session types and their mechanisations are mature [16, 6], and so are runtime monitoring of session types [3, 20], blame [18], the theory of execution monitors [26, 24], and language-based information flow [25, 5]. Applying protocols to LLM agents is recent and concurrent: Lean-verified message-sequence-chart workflows for cooperative agents [4], single-agent typestate enforced by tool masking [28], choreographies [11], BSPL enactment [19], and LLMs synthesising session-type refinements [23]. On the security side, ControlValve enforces an LLM-generated control-flow graph [17], and CaMeL and design patterns isolate untrusted data [9, 2]. Our position is the combination none of these occupies: the protocol is a *security policy* enforced against agents that may be fully compromised; the data plane gets a noninterference theorem for arbitrary agents with a certified bit bound; and LLM-written protocols are admitted only by a verified check whose counterexamples drive repair.

7 Limitations

Protocols order messages and type their fields, but do not bind the *content* of a free-text field to an earlier message: a compromised actor may fill a field the interface leaves free (our residual commit-message attacks); interfaces close this by routing such fields through an endorser. We bound *where* free text flows and *how many* attacker bits reach clean agents; we do not judge the meaning of text an endorser reads, so endorsers are an explicit trust assumption. A compromised role keeps every capability the protocol grants it; the guarantee is that it gets no more. The theorems assume complete mediation and authenticated senders; attacks outside the mediated channel (tool poisoning, model backdoors) need other defences. Liveness under adversaries is impossible for a monitor: a silent agent stalls its session. Capacities of integers ignore refinements (sound but loose). T6 (sidecars) additionally assumes a local projection per role and FIFO channels. GateBench's adaptive attacker is scripted; a learned attacker could find other paths inside the same envelope, but not outside it (T1). Opus refused 4 ops synthesis tasks; we report them as refusals. Finally, the counting step from B bits to 2^B distinguishable attacker strategies is argued on paper, not mechanised.

Ethical considerations and dual use

SwarmGate is a defensive technology: it restricts what LLM agents can do and makes their failures contained and attributable. No human subjects or personal data are involved; all tools and data in GateBench are simulated and every secret is a fake placeholder. *Dual use:* GateBench's attacker uses well-known techniques (spraying, impersonation, mimicry) with inert payloads, so it gives attackers little uplift while letting defenders measure their systems; we did not circumvent model safeguards (when Claude declined to play the attacker we scripted it, and refused synthesis tasks are reported as refusals). *Over-reliance:* the guarantees hold only under the stated assumptions and bound where content flows, not what it means; an admitted protocol is one verified layer of defence in depth, not a replacement for model safeguards, monitoring and human oversight of high-stakes actions.

References

[1] R. Alur, K. Etessami, and M. Yannakakis. Realizability and verification of MSC graphs. *Theoretical Computer Science*, 331(1):97–114, 2005. Conference version ICALP 2001, LNCS 2076, pp. 797–808.

[2] L. Beurer-Kellner, B. Buesser, A.-M. Creţu, E. Debenedetti, D. Dobos, D. Fabian, M. Fischer, D. Froelicher, K. Grosse, D. Naeff, E. Ozoani, A. Paverd, F. Tramèr, and V. Volhejn. Design patterns for securing LLM agents against prompt injections, 2025.

[3] L. Bocchi, T.-C. Chen, R. Demangeon, K. Honda, and N. Yoshida. Monitoring networks through multiparty session types. *Theoretical Computer Science*, 669:33–58, 2017.

[4] B. Bollig, M. Függer, and T. Nowak. Provable coordination for LLM agents via message sequence charts. In *Proceedings of ISoLA 2026*, 2026. ZipperGen; projection and deadlock-freedom mechanized in Lean 4.

[5] S. Capecchi, I. Castellani, and M. Dezani-Ciancaglini. Information flow safety in multiparty sessions. *Mathematical Structures in Computer Science*, 2016.

[6] D. Castro-Perez, F. Ferreira, L. Gheri, and N. Yoshida. Zooid: a DSL for certified multiparty computation: from mechanised metatheory to certified multiparty processes. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, pages 237–251, 2021.

[7] D. Castro-Perez, F. Ferreira, and S.-S. Jongmans. A synthetic reconstruction of multiparty session types. In *Proc. ACM Program. Lang. (POPL)*, 2026. arXiv:2511.22692.

[8] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025.

[9] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr. Defeating prompt injections by design, 2025.

[10] J. A. Goguen and J. Meseguer. Security policies and security models. In *IEEE Symposium on Security and Privacy*, pages 11–20, 1982.

[11] Gopinathan, Feser, Naim, Tavares, and Bingham. Pact: A choreographic language for agentic ecosystems, 2026. CP 2026 workshop (PLDI 2026). Only surnames verified; add given names from arXiv.

[12] X. Gu, X. Zheng, T. Pang, C. Du, Q. Liu, Y. Wang, J. Jiang, and M. Lin. Agent smith: A single image can jailbreak one million multimodal LLM agents exponentially fast. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 16647–16672, 2024.

[13] L. Hammond, A. Chan, J. Clifton, et al. Multi-agent risks from advanced AI. Technical Report 1, Cooperative AI Foundation, 2025.

[14] P. He, Y. Lin, S. Dong, H. Xu, Y. Xing, and H. Liu. Red-teaming LLM multi-agent systems via communication attacks. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 6726–6747, 2025.

[15] K. Honda, N. Yoshida, and M. Carbone. Multiparty asynchronous session types. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 273–284, 2008.

[16] K. Honda, N. Yoshida, and M. Carbone. Multiparty asynchronous session types. *Journal of the ACM*, 63(1):9:1–9:67, 2016.

[17] R. Jha, H. Triedman, J. Wagle, and V. Shmatikov. Breaking and fixing defenses against control-flow hijacking in multi-agent systems. In *International Conference on Learning Representations (ICLR)*, 2026.

[18] L. Jia, H. Gommerstadt, and F. Pfenning. Monitors and blame assignment for higher-order session types. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 582–594, 2016.

[19] O. Joshi, M. P. Singh, and A. K. Chopra. Ahoy: LLMs enacting multiagent interaction protocols, 2026. EMAS 2026 workshop.

[20] J. K. Larsen, A. Scalas, G. Amir, J. Jacobs, J. Wagemaker, and N. Foster. NEST: Network enforced session types. In *40th European Conference on Object-Oriented Programming (ECOOP)*, volume 372 of *LIPIcs*, pages 17:1–17:31, 2026. Lean 4 mechanisation: https://github.com/alcestes/NestLean.

[21] D. Lee and M. Tiwari. Prompt infection: LLM-to-LLM prompt injection within multi-agent systems, 2024.

[22] E. Li, F. Stutz, T. Wies, and D. Zufferey. Complete multiparty session type projection with automata. In *Computer Aided Verification (CAV 2023)*, LNCS, pages 350–373. Springer, 2023.

[23] Y. Li, P. Hou, and N. Yoshida. Specification-guided synthesis of deadlock-free communication protocol refinements with large language models. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2026. arXiv:2607.27964.

[24] J. Ligatti, L. Bauer, and D. Walker. Edit automata: Enforcement mechanisms for run-time security policies. *International Journal of Information Security*, 4(1–2):2–16, 2005.

[25] A. Sabelfeld and A. C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003.

[26] F. B. Schneider. Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000.

[27] C. Schroeder de Witt et al. Open challenges in multi-agent security: Towards secure systems of interacting AI agents, 2025.

[28] M. Sikaroudi. Concordat: Typed sessions for LLM agents – enforcing conversation protocols on a model that cannot be trusted to follow them, 2026.

[29] H. Triedman, R. Jha, and V. Shmatikov. Multi-agent systems execute arbitrary malicious code. In *Conference on Language Modeling (COLM)*, 2025.